

CS 306-COMPUTER NETWORKS

Module-III

Network layer – Routing – Shortest path routing, Flooding, Distance Vector Routing, Link State Routing, RIP, OSPF, Routing for mobile hosts.

The Network Layer:

Responsible for delivering packets between endpoints over multiple links

- Concerned with getting packets from source to destination.
- The network layer must know the topology of the subnet and choose appropriate paths through it.
- When source and destination are in *different networks*, the network layer (IP) must deal with these differences.

Design Goals:

- The services provided by the network layer should be independent of the subnet topology.
- The Transport Layer should be shielded from the number, type and topology of the subnets present.
- The network addresses available to the Transport Layer should use a uniform numbering plan (even across LANs and WANs).

Network Layer Design Issues:

- Store-and-forward packet switching
- Services provided to transport layer
- Implementation of connectionless service
- Implementation of connection-oriented service
- Comparison of virtual-circuit and datagram networks

Store-and-Forward Packet Switching:

Hosts send packets into the network; packets are forwarded by routers

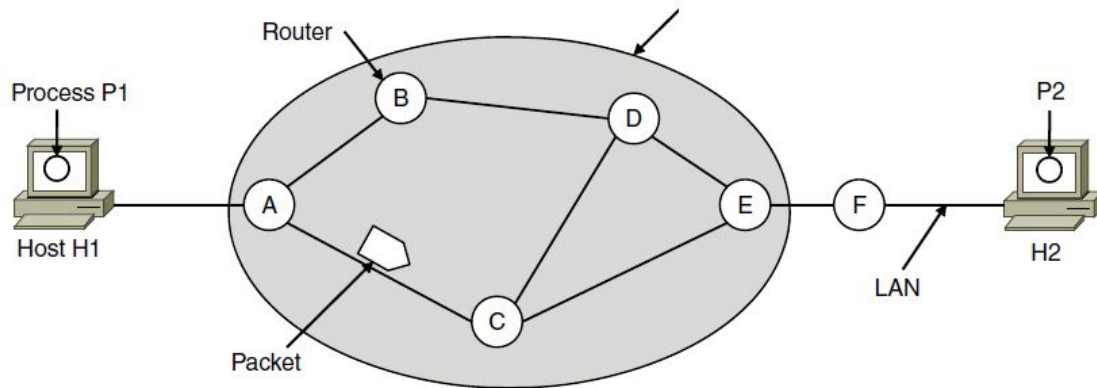


Figure: The environment of the network layer protocols.

Services Provided to the Transport Layer:

1. Services independent of router technology.
2. Transport layer shielded from number, type, topology of routers.
3. Network addresses available to transport layer use uniform numbering plan even across LANs and WANs

Implementation of Connectionless Service:

- Datagram
- Virtual Circuit

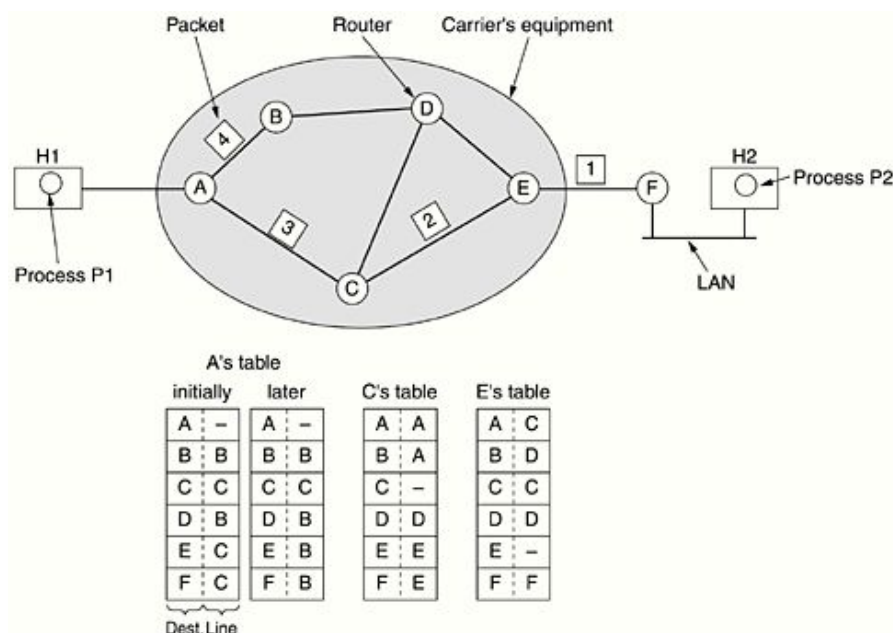


Figure: Routing within a datagram subnet

Implementation of Connection-Oriented Service:

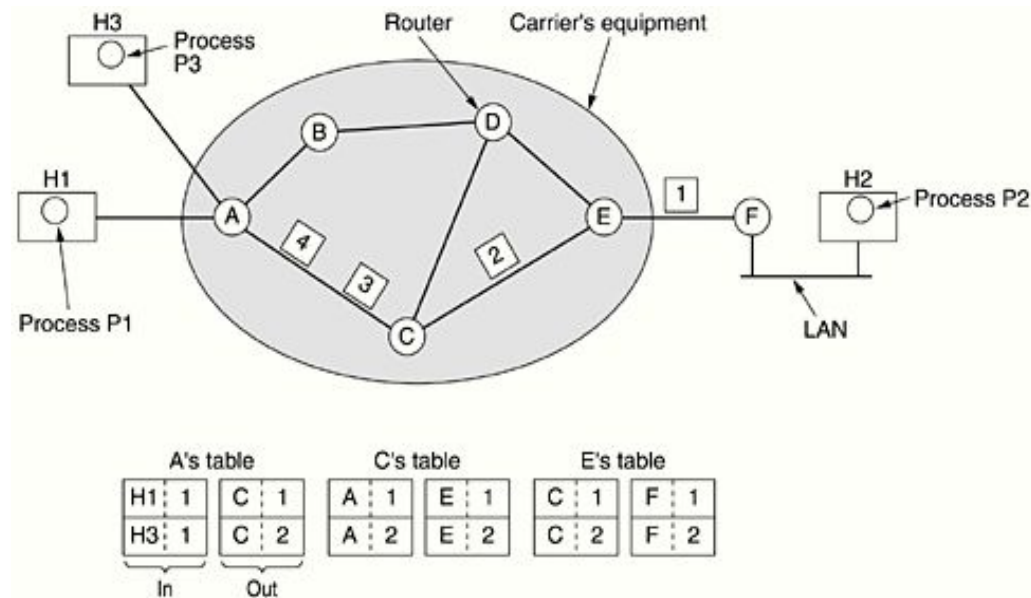


Figure: Routing within a virtual-circuit subnet.

Comparison of Virtual-Circuit and Datagram Networks:

Issue	Datagram subnet	Virtual-circuit subnet
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC

Routing Algorithms:

The routing algorithm is that part of the network layer software responsible for deciding which output line an incoming packet should be transmitted on.

- Static Routing
- Dynamic Routing
 - Optimality principle
 - **Shortest path algorithm**
 - **Flooding**
 - **Distance vector routing**
 - **Link state routing**
 - Hierarchical Routing
 - Broadcast Routing
 - Multicast Routing

- **Routing for Mobile Hosts**
- Routing in ad hoc networks

Optimality Principle:

Before we get into specific algorithms, it may be helpful to note that one can make a general statement about optimal routes without regard to network topology or traffic. This statement is known as the **optimality principle**.

As a direct consequence of the optimality principle, we can see that the set of optimal routes from all sources to a given destination form a tree rooted at the destination. Such a tree is called a **sink tree** and is illustrated in Fig. , where the distance metric is the number of hops. Note that a sink tree is not necessarily unique; other trees with the same path lengths may exist. The goal of all routing algorithms is to discover and use the sink trees for all routers.

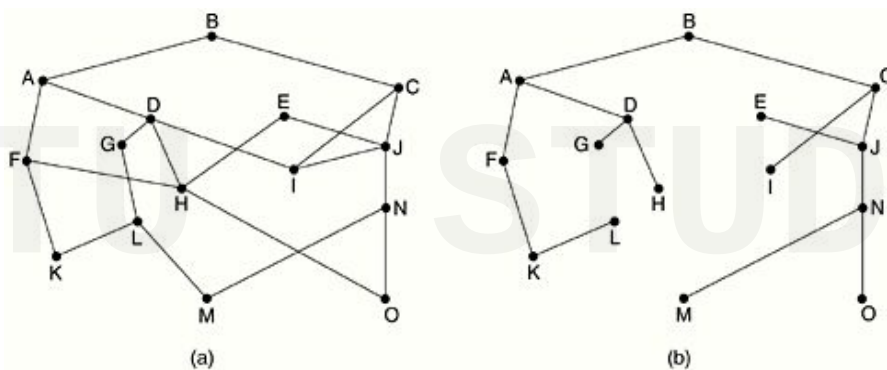


Figure: (a) A subnet. (b) A sink tree for router

Since a sink tree is indeed a tree, it does not contain any loops, so each packet will be delivered within a finite and bounded number of hops. In practice, life is not quite this easy. Links and routers can go down and come back up during operation, so different routers may have different ideas about the current topology. Also, we have quietly finessed the issue of whether each router has to individually acquire the information on which to base its sink tree computation or whether this information is collected by some other means. We will come back to these issues shortly. Nevertheless, the optimality principle and the sink tree provide a benchmark against which other routing algorithms can be measured.

Shortest Path Routing:

The idea is to build a graph of the subnet, with each node of the graph representing a router and each arc of the graph representing a communication line (often called a link). To choose a route between a given pair of routers, the algorithm just finds the shortest path between them on the graph.

The concept of a **shortest path** deserves some explanation. One way of measuring path length is the number of hops. Using this metric, the paths *ABC* and *ABE* in Fig. are equally long. Another metric is the geographic distance in kilometers, in which case *ABC* is clearly much longer than *ABE* (assuming the figure is drawn to scale).

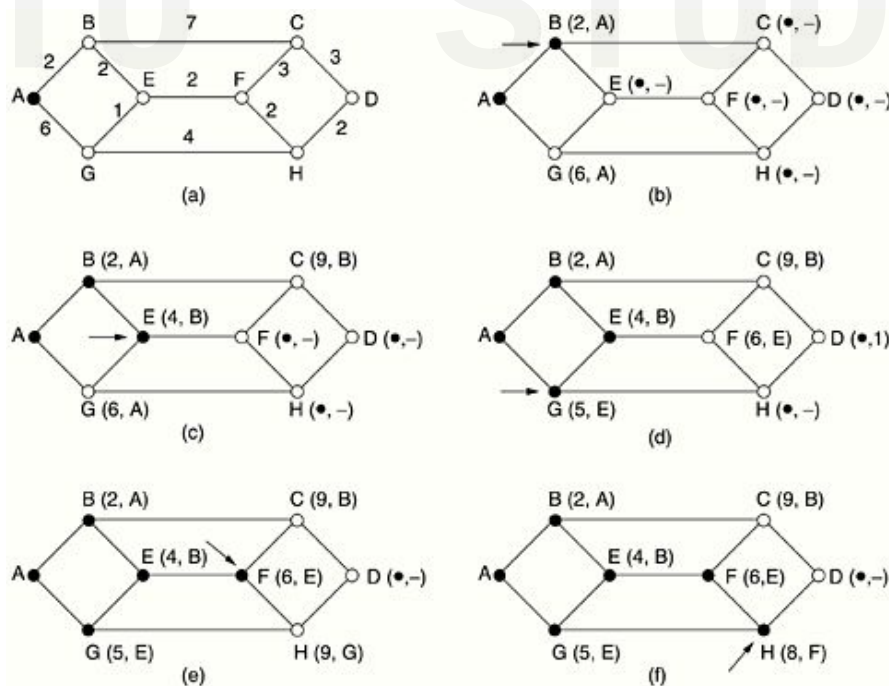


Figure: The first five steps used in computing the shortest path from A to D. The arrows indicate the working node.

Several algorithms for computing the shortest path between two nodes of a graph are known. This one is due to Dijkstra (1959). Each node is labeled (in parentheses) with its distance from the source node along the best known path. Initially, no paths are known, so all nodes are relabeled with infinity. As the algorithm proceeds and paths are found, the labels may change, reflecting better paths. A label may be either tentative or permanent. Initially, all labels are tentative. When it is discovered that a label represents the shortest possible path from the source to that node, it is made permanent and never changed thereafter. To illustrate how the labeling algorithm works, look at the weighted, undirected graph of Fig.(a), where the weights represent, for example, distance. We want to find the shortest path from A to D . We start out by marking node A as permanent, indicated by a filled-in circle. Then we examine, in turn, each of the nodes adjacent to A (the working node), relabeling each one with the distance to A . Whenever a node is relabeled, we also label it with the node from which the probe was made so that we can reconstruct the final path later. Having examined each of the nodes adjacent to A , we examine all the tentatively labeled nodes in the whole graph and make the one with the smallest label permanent, as shown in Fig. (b). This one becomes the new working node. We now start at B and examine all nodes adjacent to it. If the sum of the label on B and the distance from B to the node being considered is less than the label on that node, we have a shorter path, so the node is relabeled.

After all the nodes adjacent to the working node have been inspected and the tentative labels changed if possible, the entire graph is searched for the tentatively-labeled node with the smallest value. This node is made permanent and becomes the working node for the next round. Figure shows the first five steps of the algorithm. To see why the algorithm works, look at Fig.(c). At that point we have just made E permanent. Suppose that there were a shorter path than ABE , say $AXYZE$. There are two possibilities: either node Z has already been made permanent, or it has not been. If it has, then E has already been probed (on the round following the one when Z was made permanent), so the $AXYZE$ path has not escaped our attention and thus cannot be a shorter path. Now consider the case where Z is still tentatively labeled. Either the label at Z is greater than or equal to that at E , in which case $AXYZE$ cannot be a shorter path than ABE , or it is less than that of E , in which case Z and not E will become permanent first, allowing E to be probed from Z .

Dijkstra's algorithm to compute the shortest path through a graph:

```
#define MAX_NODES 1024          /* maximum number of nodes */
#define INFINITY 1000000000     /* a number larger than every maximum path */
int n, dist[MAX_NODES][MAX_NODES]; /* dist[i][j] is the distance from i to j */
void shortest_path(int s, int t, int path[])
{
    struct state {
        int predecessor;          /* the path being worked on */
        int length;               /* previous node */
        enum {permanent, tentative} label; /* length from source to this node */
    } state[MAX_NODES];
    int i, k, min;
    struct state *p;
    for (p = &state[0]; p < &state[n]; p++) { /* initialize state */
        p->predecessor = -1;
        p->length = INFINITY;
        p->label = tentative;
    }
    state[t].length = 0; state[t].label = permanent;
    k = t;
    do {
        /* Is there a better path from k? */
        for (i = 0; i < n; i++) /* this graph has n nodes */
            if (dist[k][i] != 0 && state[i].label == tentative) {
                if (state[k].length + dist[k][i] < state[i].length) {
                    state[i].predecessor = k;
                    state[i].length = state[k].length + dist[k][i];
                }
            }

        /* Find the tentatively labeled node with the smallest label. */
        k = 0; min = INFINITY;
        for (i = 0; i < n; i++)
            if (state[i].label == tentative && state[i].length < min) {
                min = state[i].length;
                k = i;
            }
        state[k].label = permanent;
    } while (k != s);

    /* Copy the path into the output array. */
    i = 0; k = s;
    do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}
```

Flooding:

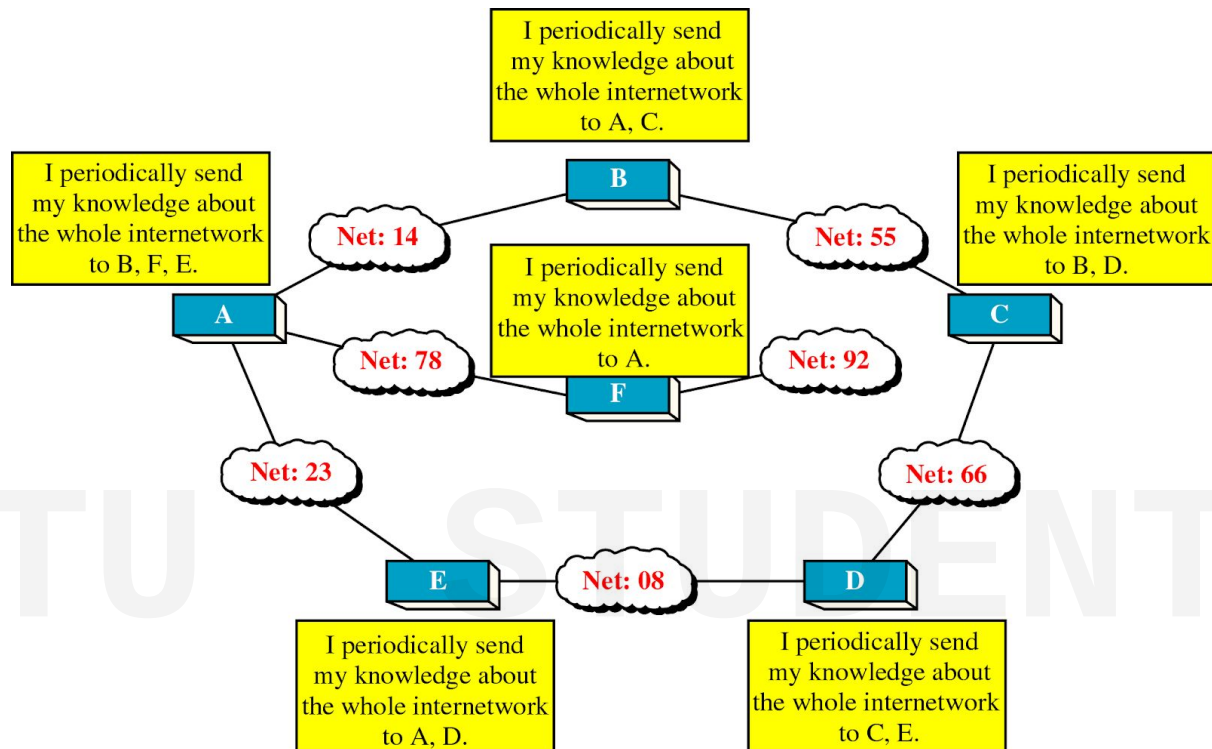
Flooding, in which every incoming packet is sent out on every outgoing line except the one it arrived on. Flooding obviously generates vast numbers of duplicate packets, in fact, an infinite number unless some measures are taken to damp the process. One such measure is to have a hop counter contained in the header of each packet, which is decremented at each hop, with the packet being discarded when the counter reaches zero. Ideally, the hop counter should be initialized to the length of the path from source to destination. If the sender does not know how long the path is, it can initialize the counter to the worst case, namely, the full diameter of the subnet

Distance Vector Routing:

Distance vector routing algorithms operate by having each router maintain a table (i.e, a vector) giving the best known distance to each destination and which line to use to get there.

These tables are updated by exchanging information with the neighbors. The distance vector routing algorithm is sometimes called by other names, most commonly the distributed **Bellman-Ford** routing algorithm and the **Ford-Fulkerson** algorithm, after the researchers who developed it (Bellman, 1957; and Ford and Fulkerson, 1962). It was the original ARPANET routing algorithm and was also used in the Internet under the name RIP.

Concept of Distance Vector Routing:



In distance vector routing, each router maintains a routing table indexed by, and containing one entry for, each router in the subnet. This entry contains two parts: the preferred outgoing line to use for that destination and an estimate of the time or distance to that destination. The metric used might be number of hops, time delay in milliseconds, total number of packets queued along the path, or something similar.

Part (a) shows a subnet. The first four columns of part (b) show the delay vectors received from the neighbors of router J. A claims to have a 12-msec delay to B, a 25-msec delay to C, a 40-msec delay to D, etc. Suppose that J has measured or estimated its delay to its neighbors, A, I, H, and K as 8, 10, 12, and 6 msec, respectively.

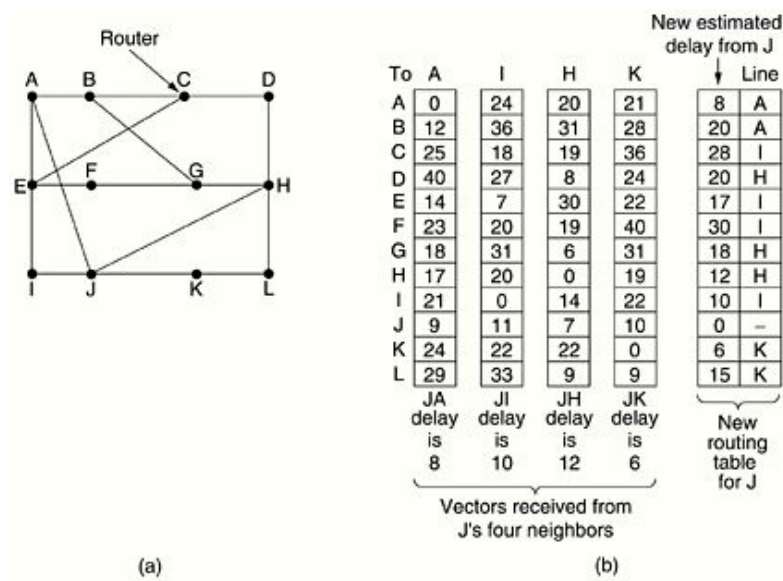


Figure:
(a) A subnet. (b) Input from

A

I

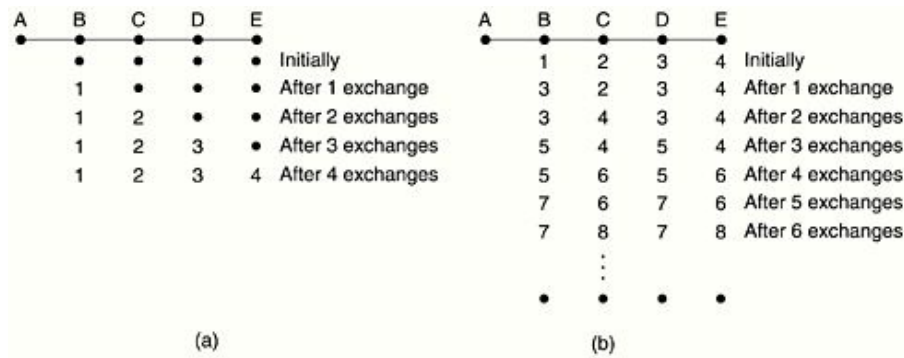
H

’
K

, and the new routing table for

J

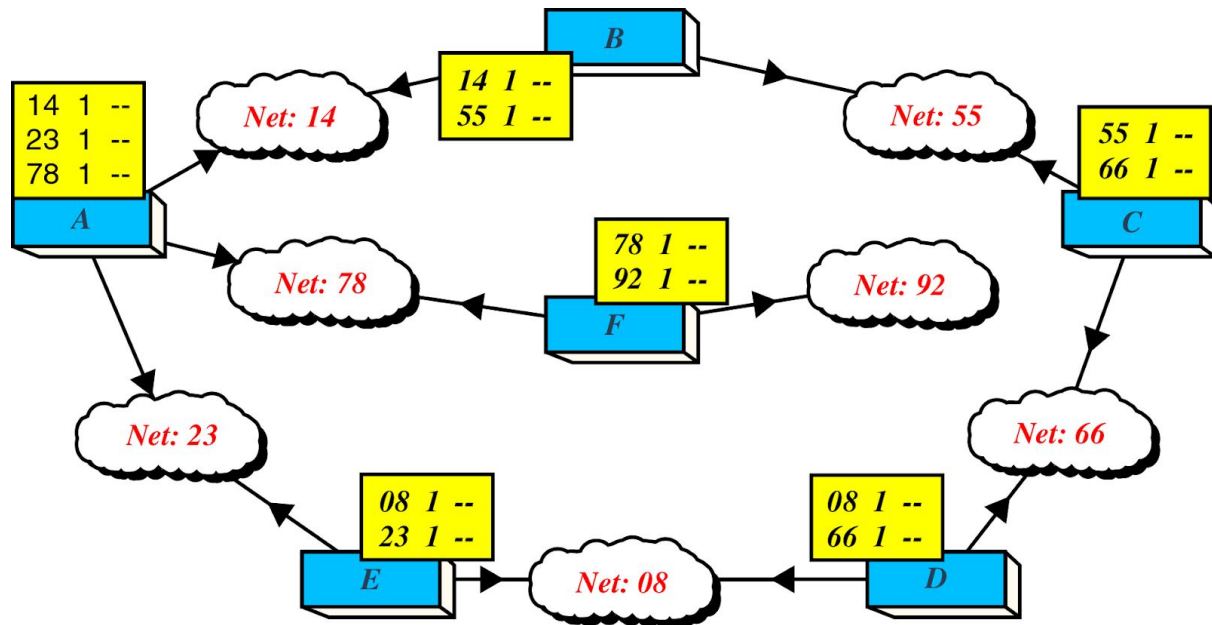
Consider how J computes its new route to router G . It knows that it can get to A in 8 msec, and A claims to be able to get to G in 18 msec, so J knows it can count on a delay of 26 msec to G if it forwards packets bound for G to A . Similarly, it computes the delay to G via I , H , and K as 41 ($31 + 10$), 18 ($6 + 12$), and 37 ($31 + 6$) msec, respectively. The best of these values is 18, so it makes an entry in its routing table that the delay to G is 18 msec and that the route to use is via H . The same calculation is performed for all the other destinations, with the new routing table shown in the last column of the figure



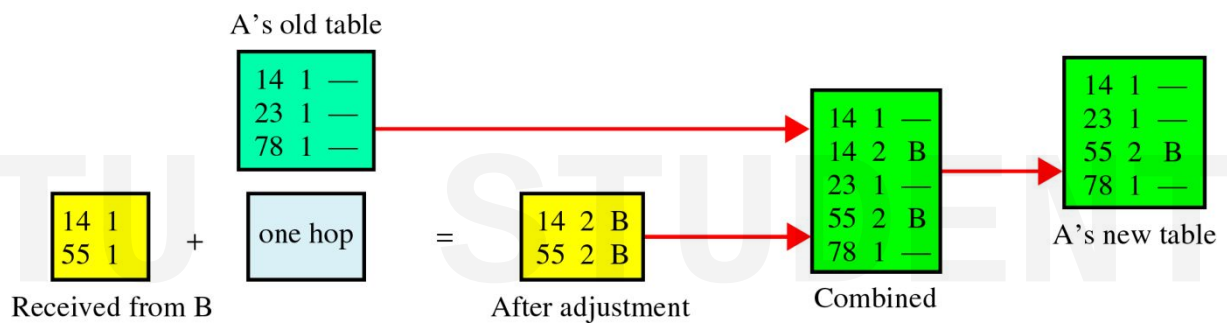
Distance Vector Routing Table:

Network ID	Cost	Next Hop
• • • • • • • • • •	• • • • • • • •	• • • • • • • • • •
• • • • • • • • • •	• • • • • • • •	• • • • • • • • • •
• • • • • • • • • •	• • • • • • • •	• • • • • • • • • •
• • • • • • • • • •	• • • • • • • •	• • • • • • • • • •
• • • • • • • • • •	• • • • • • • •	• • • • • • • • • •

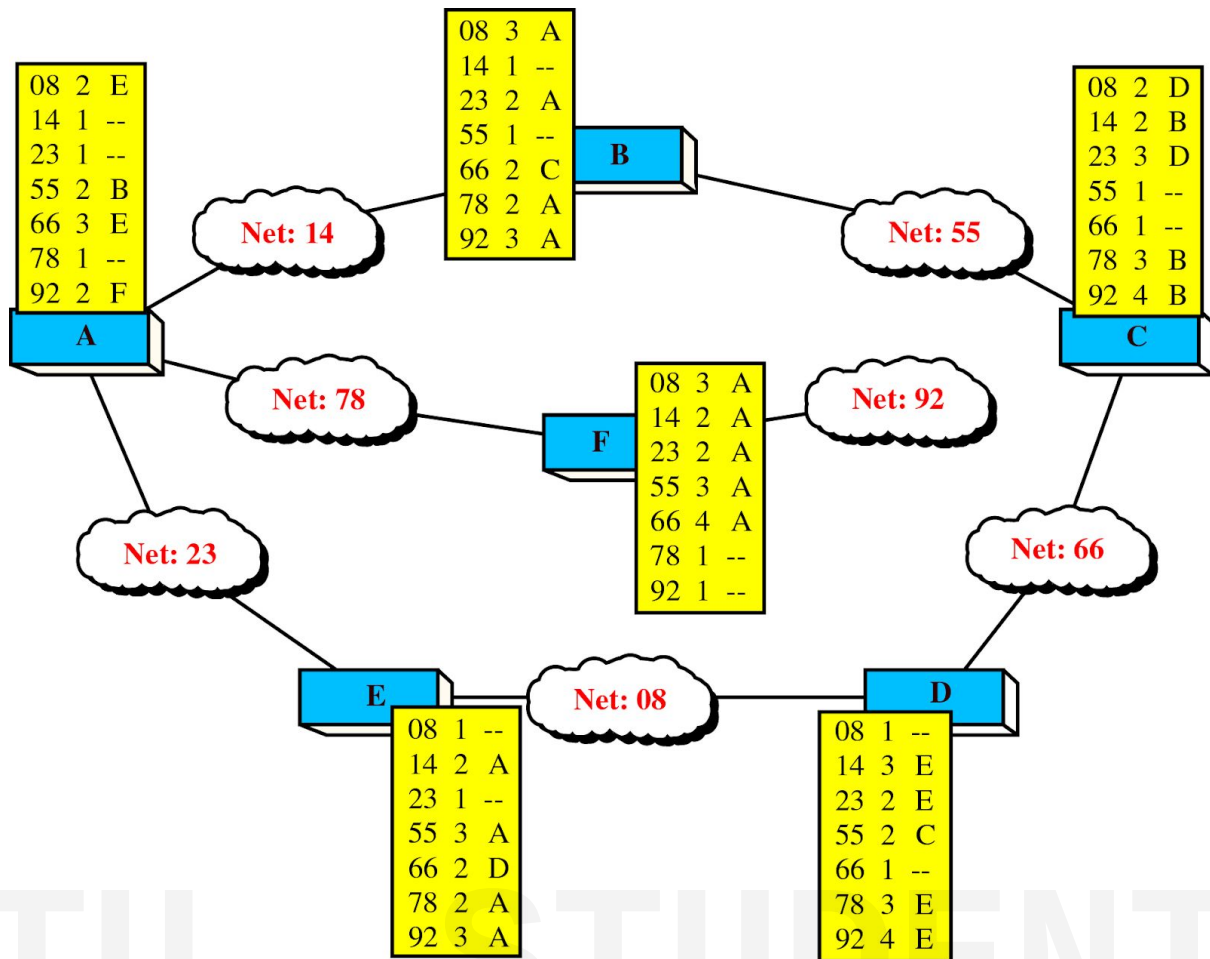
Routing table Distribution:



Updating Routing Table for Router A:



Final Routing Table:



Count-to-Infinity Problem:

From this figure, it should be clear why bad news travels slowly: no router ever has a value more than one higher than the minimum of all its neighbors. Gradually, all routers work their way up to infinity, but the number of exchanges required depends on the numerical value used for infinity. For this reason, it is wise to set infinity to the longest path plus 1. If the metric is time delay, there is no well-defined upper bound, so a high value is needed to prevent a path with a long delay from being treated as down. Not entirely surprisingly, this problem is known as the **count-to-infinity** problem.

Link State Routing:

Variants of link state routing are now widely used. The idea behind link state routing is simple and can be stated as five parts. Each router must do the following:

1. Discover its neighbors and learn their network addresses.

2. Measure the delay or cost to each of its neighbors.
3. Construct a packet telling all it has just learned.
4. Send this packet to all other routers.
5. Compute the shortest path to every other router.

In effect, the complete topology and all delays are experimentally measured and distributed to every router. Then Dijkstra's algorithm can be run to find the shortest path to every other router.

Learning about the Neighbors:

When a router is booted, its first task is to learn who its neighbors are. It accomplishes this goal by sending a special HELLO packet on each point-to-point line. The router on the other end is expected to send back a reply telling who it is. These names must be globally unique because when a distant router later hears that three routers are all connected to F , it is essential that it can determine whether all three mean the same

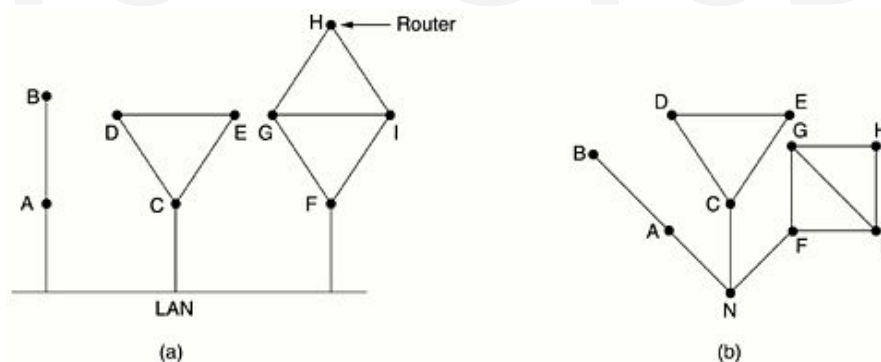
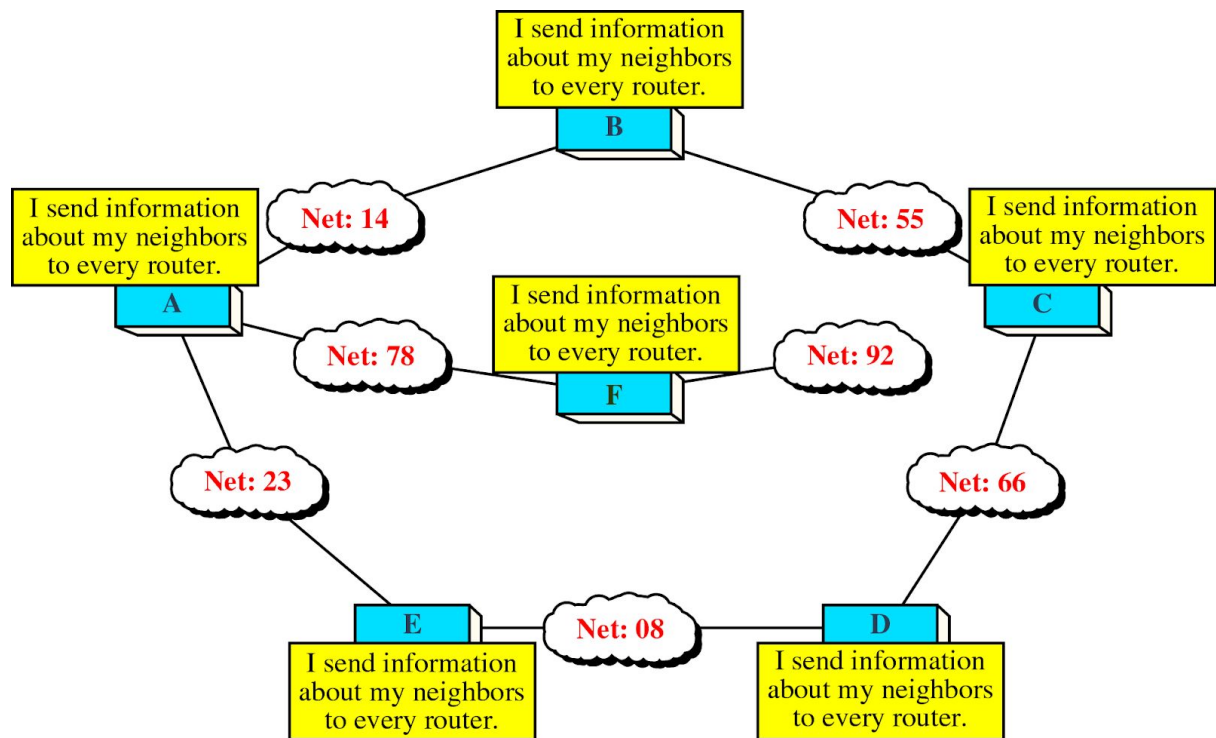
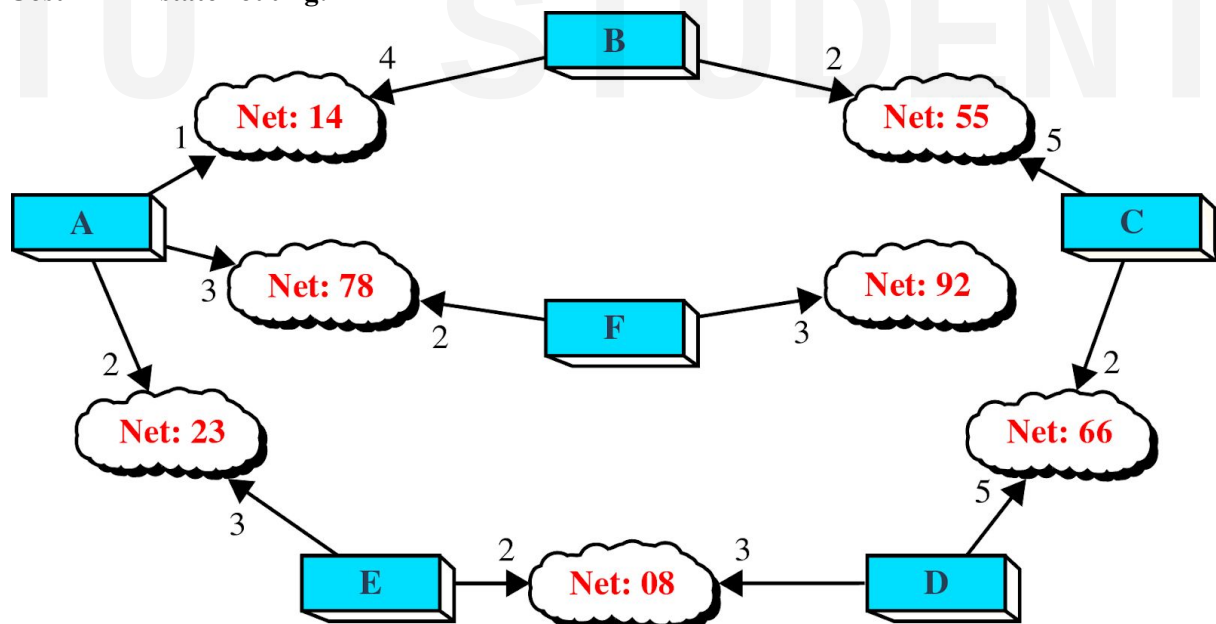


Figure . (a) Nine routers and a LAN. (b) A graph model of (a).

Concept of Link State Routing:



Cost in link state routing:



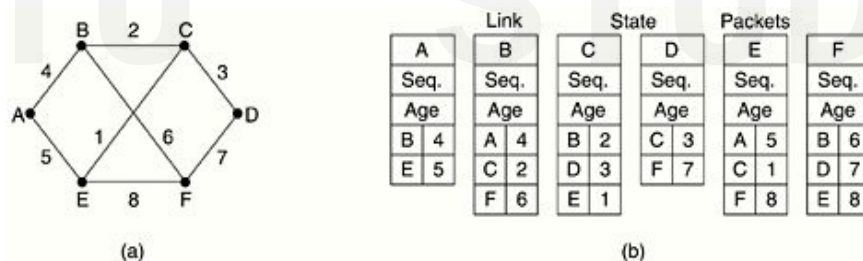
Link state packet:

Advertiser	Network	Cost	Neighbor
.			
.			
.			

Building Link State Packets:

Building the link state packets is easy. The hard part is determining when to build them. One possibility is to build them periodically, that is, at regular intervals. Another possibility is to build them when some significant event occurs, such as a line or neighbor going down or coming back up again or changing its properties appreciably

Figure (a) A subnet. (b) The link state packets for this subnet



Source	Seq.	Age	Send flags			ACK flags			Data
			A	C	F	A	C	F	
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

Figure: The packet buffer for router

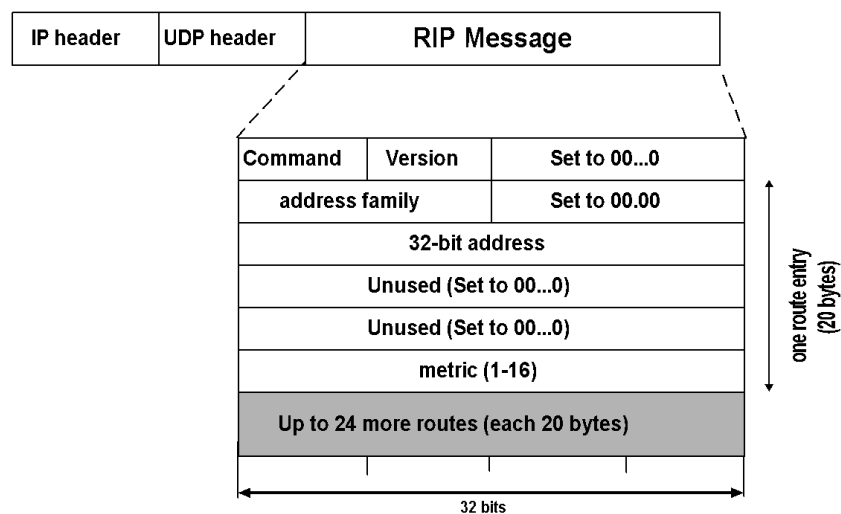
Distributing the Link State Packets:

The trickiest part of the algorithm is distributing the link state packets reliably. As the packets are distributed and installed, the routers getting the first ones will change their routes. Consequently, the different routers may be using different versions of the topology, which can lead to inconsistencies, loops, unreachable machines, and other problems.

RIP:

- A simple intradomain protocol
- Straightforward implementation of Distance Vector Routing
- Each router advertises its distance vector every 30 seconds (or whenever its routing table changes) to all of its neighbors
- RIP always uses 1 as link metric
- Maximum hop count is 15, with “16” equal to “ ∞ ”
- Routes are timeout (set to 16) after 3 minutes if they are not updated

RIP Message format:



RIP Messages:

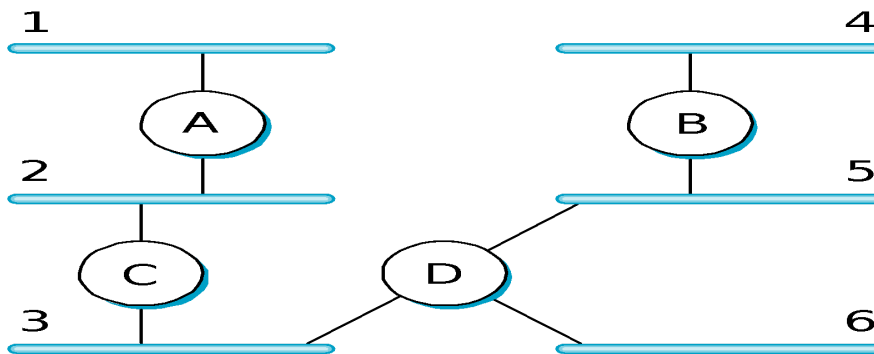
- Two types of messages:
 - **Request messages**
 - used to ask neighboring nodes for an update
 - **Response messages**
 - contains an update

RIP Problems:

- RIP takes a long time to stabilize
 - Even for a small network, it takes several minutes until the routing tables have settled after a change
- RIP has all the problems of distance vector algorithms, e.g., count-to-Infinity
 - RIP uses split horizon to avoid count-to-infinity
- The maximum path in RIP is 15 hops

An Example of RIP:

- Routers advertise the cost of reaching networks.
- In this example, C's update to A would indicate that C can reach Networks 2 and 3 with cost 0, Networks 5 and 6 with cost 1 and Network 4 with cost 2.



OSPF:

OSPF is a standardized Link-State routing protocol, designed to scale efficiently to support larger networks. OSPF adheres to the following Link State characteristics:

- OSPF employs a hierarchical network design using Areas.
- OSPF will form neighbor relationships with adjacent routers in the same Area.

- Instead of advertising the distance to connected networks, OSPF advertises the status of directly connected links using Link-State Advertisements (LSAs).
- OSPF sends updates (LSAs) when there is a change to one of its links, and will only send the change in the update. LSAs are additionally refreshed every 30 minutes.
- OSPF traffic is multicast either to address 224.0.0.5 (all OSPF routers) or 224.0.0.6 (all Designated Routers).
- OSPF uses the Dijkstra Shortest Path First algorithm to determine the shortest path.

Other characteristics of OSPF include:

- OSPF supports only IP routing.
- OSPF routes have an administrative distance is 110.
- OSPF uses cost as its metric, which is computed based on the bandwidth of the link.

OSPF has no hop-count limit.

The OSPF process builds and maintains three separate tables:

- A neighbor table – contains a list of all neighboring routers.
- A topology table – contains a list of all possible routes to all known networks within an area.
- A routing table – contains the best route for each known network.

OSPF Neighbors OSPF forms neighbor relationships, called adjacencies, with other routers in the same Area by exchanging Hello packets to multicast address 224.0.0.5. Only after an adjacency is formed can routers share routing information. Each OSPF router is identified by a unique Router ID. The Router ID can be determined in one of three ways:

- The Router ID can be manually specified.
- If not manually specified, the highest IP address configured on any Loopback interface on the router will become the Router ID.
- If no loopback interface exists, the highest IP address configured on any Physical interface will become the Router ID.

Multicast Routing:

Some applications require that widely-separated processes work together in groups, for example, a group of processes implementing a distributed database system. In these situations, it is frequently necessary for one process to send a message to all the other

members of the group. If the group is small, it can just send each other member a point-to-point message. If the group is large, this strategy is expensive. Sometimes broadcasting can be used, but using broadcasting to inform 1000 machines on a million-node network is inefficient because most receivers are not interested in the message (or worse yet, they are definitely interested but are not supposed to see it). Thus, we need a way to send messages to well-defined groups that are numerically large in size but small compared to the network as a whole. Sending a message to such a group is called **multicasting**, and its routing algorithm is called **multicast routing**.

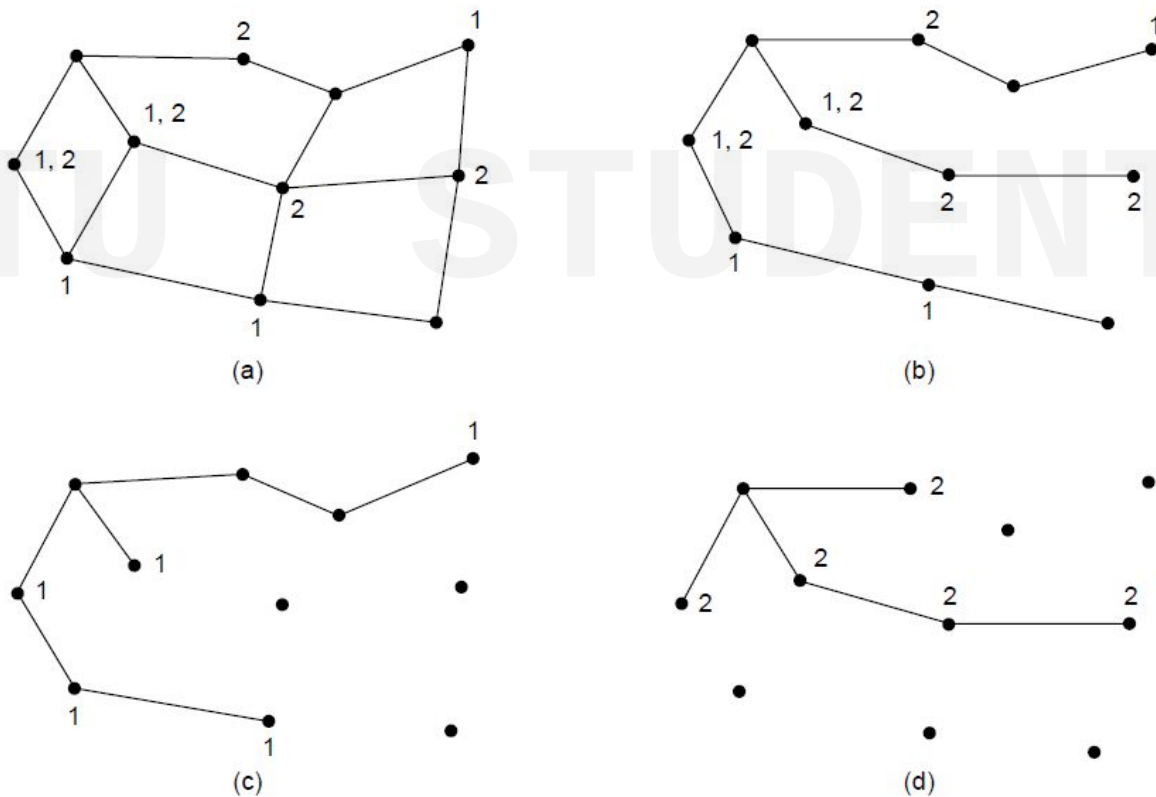
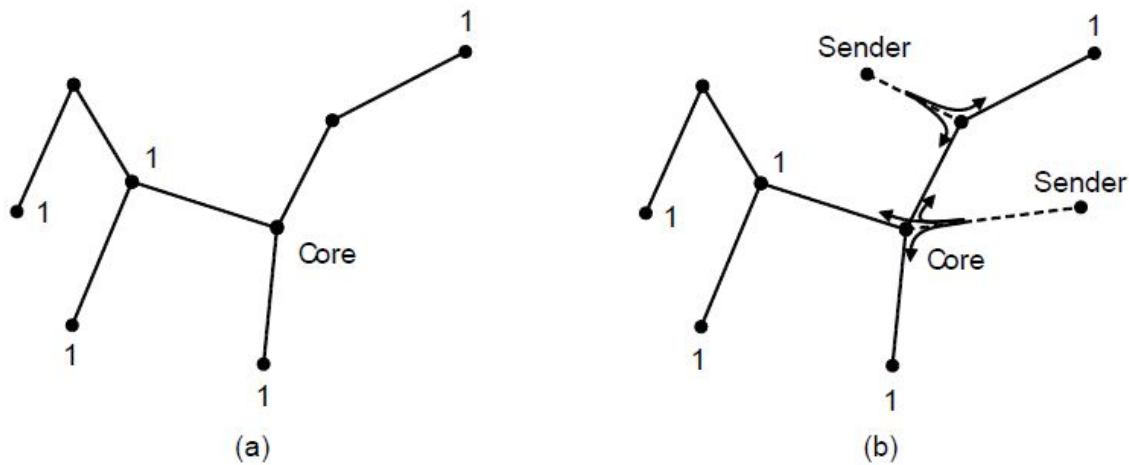


Figure: (a) A network. (b) A spanning tree for the leftmost router. (c) A multicast tree for group 1. (d) A multicast tree for group 2.



(a) Core-based tree for group 1.

(b) Sending to group 1.

Routing for Mobile Hosts:

Hosts that never move are said to be stationary. They are connected to the network by copper wires or fiber optics. In contrast, we can distinguish two other kinds of hosts. Migratory hosts are basically stationary hosts who move from one fixed site to another from time to time but use the network only when they are physically connected to it. Roaming hosts actually compute on the run and want to maintain their connections as they move around. We will use the term **mobile hosts** to mean either of the latter two categories, that is, all hosts that are away from home and still want to be connected. All hosts are assumed to have a permanent **home location** that never changes. Hosts also have a permanent home address that can be used to determine their home locations, analogous to the way the telephone number 1-212-5551212 indicates the United States (country code 1) and Manhattan (212). The routing goal in systems with mobile hosts is to make it possible to send packets to mobile hosts using their home addresses and have the packets efficiently reach them wherever they may be. The trick, of course, is to find them.

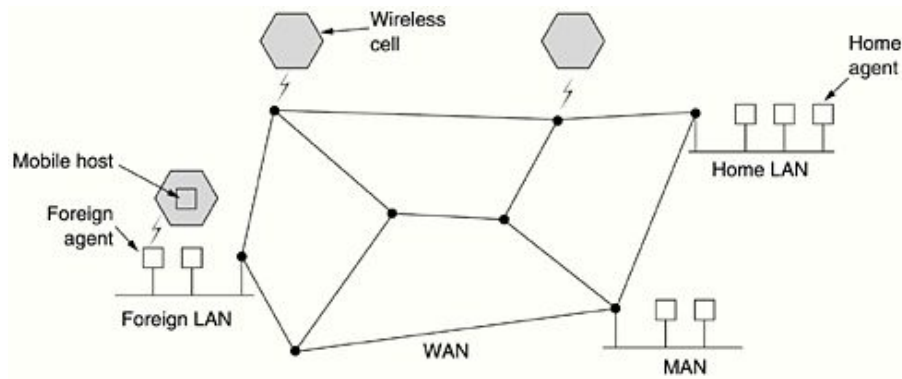
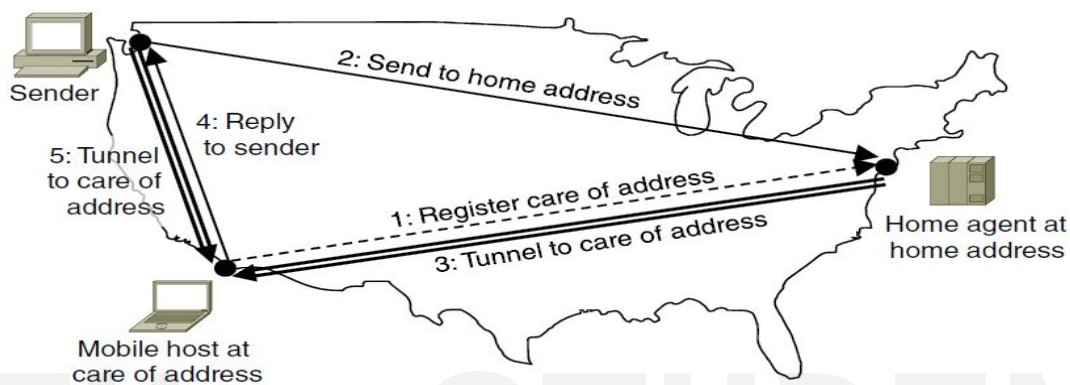


Figure: A WAN to which LANs, MANs, and wireless cells are attached.



Registration Procedure:

1. Periodically, each foreign agent broadcasts a packet announcing its existence and address. A newly-arrived mobile host may wait for one of these messages, but if none arrives quickly enough, the mobile host can broadcast a packet saying: Are there any foreign agents around?
2. The mobile host registers with the foreign agent, giving its home address, current datalink layer address, and some security information
3. The foreign agent contacts the mobile host's home agent and says: One of your hosts is over here. The message from the foreign agent to the home agent contains the foreign agent's network address. It also includes the security information to convince the home agent that the mobile host is really there.

4. The home agent examines the security information, which contains a timestamp, to prove that it was generated within the past few seconds. If it is happy, it tells the foreign agent to proceed.
 5. When the foreign agent gets the acknowledgement from the home agent, it makes an entry in its tables and informs the mobile host that it is now registered.
-

KTU STUDENTS



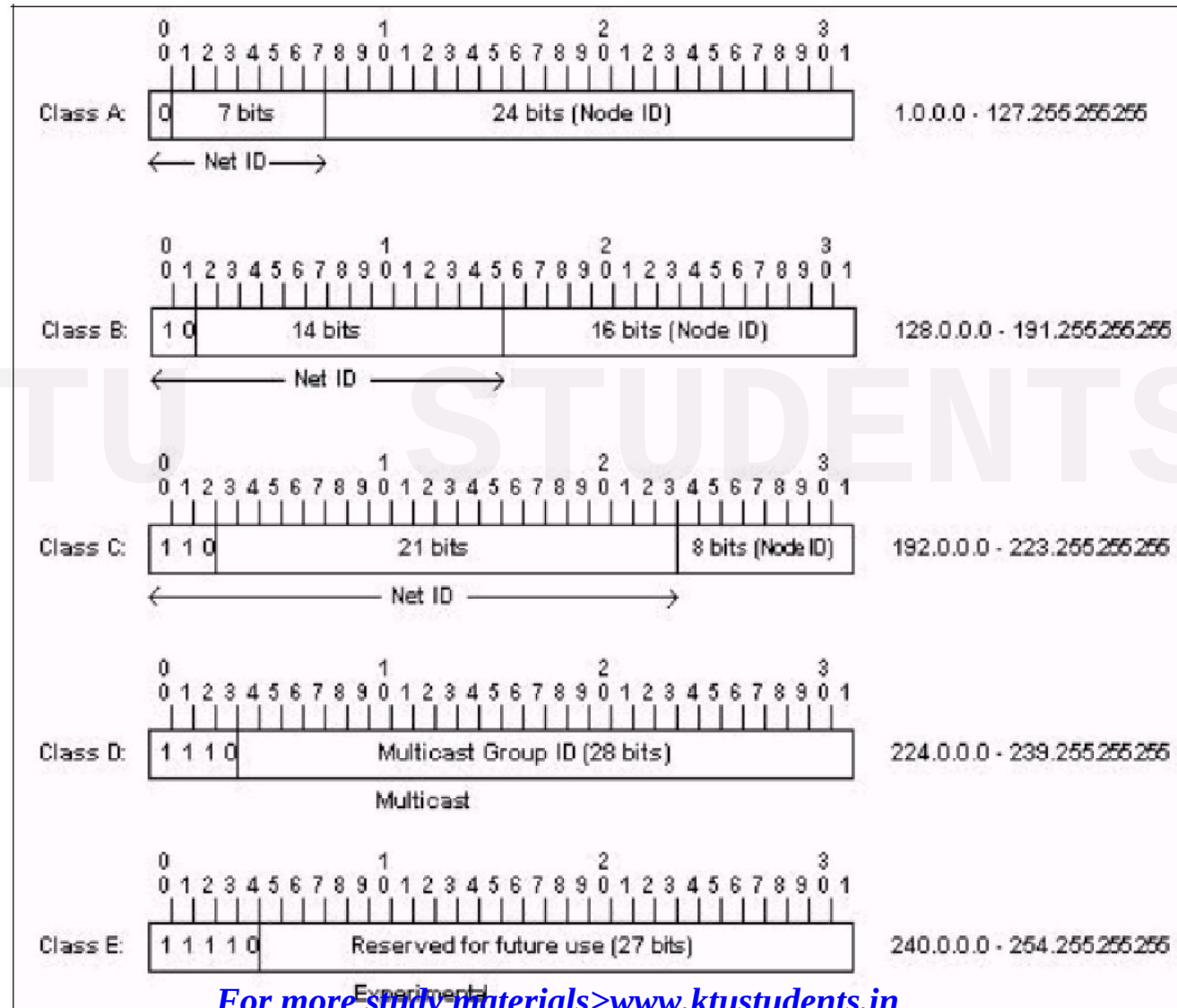
IP Addressing & Subnetting

- IP Addressing
- Subnetting
- VLSM
- CIDR

IP Addresses

- An IP address is an address used to uniquely identify a device on an IP network.
- The address is made up of 32 binary bits which can be divisible into a network portion and host portion with the help of a subnet mask.
- 32 binary bits are broken into four octets (1 octet = 8 bits)
- Dotted decimal format (for example, 172.16.81.100)

IP Address Classes



IP Address Classes

- Class A: The first octet is the network portion. Octets 2, 3, and 4 are for subnets/hosts
- Class B: The first two octets are the network portion. Octets 3 and 4 are for subnets/hosts
- Class C: The first three octets are the network portion. Octet 4 is for subnets/hosts

Private Address Range

Address Class	Reserved Address Space
Class A	10.0.0.0 - 10.255.255.255
Class B	172.16.0.0 - 172.31.255.255
Class C	192.168.0.0 - 192.168.255.255

KTU STUDENTS

Network Masks

- Distinguishes which portion of the address identifies the network and which portion of the address identifies the node.
- Default masks:
 - Class A: 255.0.0.0
 - Class B: 255.255.0.0
 - Class C: 255.255.255.0

Subnetting

- Creates multiple logical networks that exist within a single Class A, B, or C network.
- If you do not subnet, you will only be able to use one network from your Class A, B, or C network, which is unrealistic
- Each data link on a network must have a unique network ID, with every node on that link being a member of the same network

Benefits of Subnetting

- 1) Reduced network traffic
- 2) Optimized network performance
- 3) Simplified management
- 4) Facilitated spanning of large geographical distances

IP Subnet-Zero

- This command allows you to use the first and last subnet in your network design.
- For example, the Class C mask of 192 provides subnets 64 and 128, but with the IP subnet-zero command, you now get to use subnets 0, 64, 128, and 192

How to create subnets

- Determine the number of required network IDs:
 - One for each subnet
 - One for each wide area network connection
- Determine the number of required host IDs per subnet:
 - One for each TCP/IP host
 - One for each router interface
- Based on the above requirements, create the following:
 - One subnet mask for your entire network
 - A unique subnet ID for each physical segment
 - A range of host IDs for each subnet

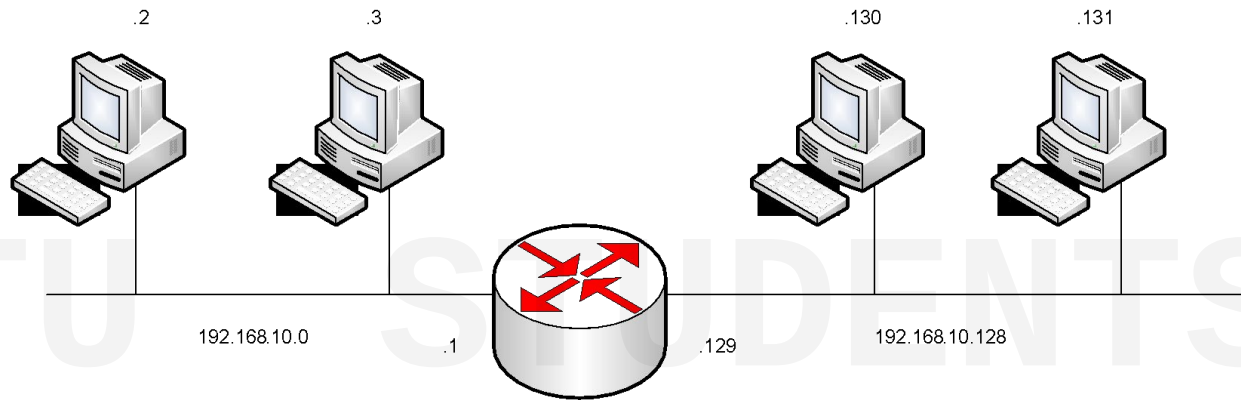
Subnetting a Class A/B/C Address

- How many subnets does the chosen subnet mask produce?
- How many valid hosts per subnet are available?
- What are the valid subnets?
- What's the broadcast address of each subnet?
- What are the valid hosts in each subnet?

Practice Example #1C: 255.255.255.128 (/25) Network 192.168.10.0

- How many subnets? Since 128 is 1 bit on (100000000), the answer would be $2^1 = 2$.
- How many hosts per subnet? We have 7 host bits off (100000000), so the equation would be $2^7 - 2 = 126$ hosts.
- What are the valid subnets? $256 - 128 = 128$. Remember, we'll start at zero and count in our block size, so our subnets are 0, 128.
- What's the broadcast address for each subnet? The number right before the value of the next subnet is all host bits turned on and equals the broadcast address. For the zero subnet, the next subnet is 128, so the broadcast of the 0 subnet is 127.
- What are the valid hosts? These are the numbers between the subnet and broadcast address

Logical Network Implementation



Practice Example #2C: 255.255.255.224 (/27) Network 192.168.10.0

- How many subnets? 224 is 11100000, so our equation would be $2^3 = 8$.
- How many hosts? $2^5 - 2 = 30$.
- What are the valid subnets? $256 - 224 = 32$.
We just start at zero and count to the subnet mask value in blocks (increments) of 32: 0, 32, 64, 96, 128, 160, 192, and 224.
- What's the broadcast address for each subnet (always the number right before the next subnet)?
- What are the valid hosts (the numbers between the subnet number and the broadcast address)?

Practice Example #2C: 255.255.255.224 (/27) Network 192.168.10.0

Subnet Address	0	32		192	224
First Host	1	33		193	225
Last Host	30	62		222	254
Broadcast Address	31	63		223	255

Practice Example #1B: 255.255.128.0 (/17) Network 172.16.0.0

- Subnets? $2^1 = 2$
- Hosts? $2^{15} - 2 = 32,766$ (7 bits in the third octet, and 8 in the fourth)
- Valid subnets? $256 - 128 = 128$. 0, 128.
Remember that subnetting is performed in the third octet, so the subnet numbers are really 0.0 and 128.0, as shown in the next table
- Broadcast address for each subnet?
- Valid hosts?

Practice Example #1B: 255.255.128.0 (/17) Network 172.16.0.0

Subnet	0.0	128.0
First Host	0.1	128.1
Last Host	127.254	255.254
Broadcast	127.255	255.255

KTU STUDENTS

Practice Example #2B: 255.255.240.0 (/20) Network 172.16.0.0

- Subnets? $2^4 = 16$.
- Hosts? $2^{12} - 2 = 4094$.
- Valid subnets? $256 - 240 = 0, 16, 32, 48$, etc., up to 240.
- Broadcast address for each subnet?
- Valid hosts?

Practice Example #2B: 255.255.240.0 (/20) Network 172.16.0.0

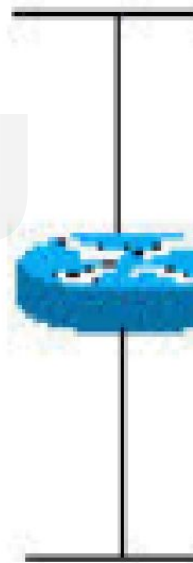
Subnet	0.0	16.0		240.0
First Host	0.1	16.1		240.1
Last Host	15.254	31.254		255.254
Broadcast	15.255	31.255		255.255

Variable Length Subnet Mask(VLSM)

Subnet with requirements shown?

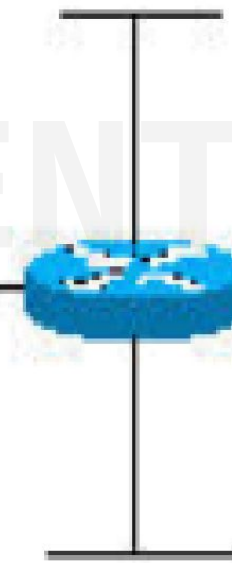
netA: 14 hosts

netD: 7 hosts



netB: 28 hosts

netC: 2 hosts



netE: 28 hosts

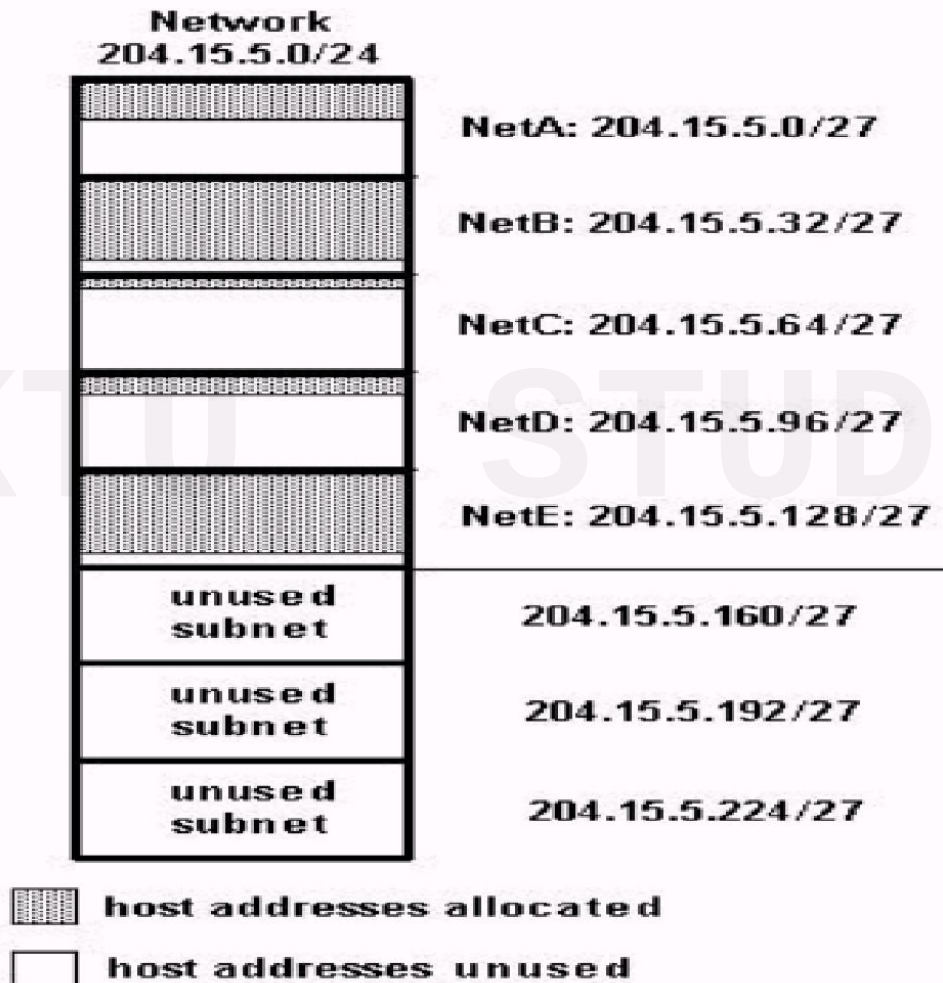
Variable Length Subnet Mask(VLSM)

Subnet with requirements shown?

- 5 subnets needed
- Can be assigned as follows:
 - netA: 204.15.5.0/27 host address range 1 to 30
 - netB: 204.15.5.32/27 host address range 33 to 62
 - netC: 204.15.5.64/27 host address range 65 to 94
 - netD: 204.15.5.96/27 host address range 97 to 126
 - netE: 204.15.5.128/27 host address range 129 to 158

Variable Length Subnet Mask(VLSM)

Subnet with requirements shown?



Variable Length Subnet Mask(VLSM)

Subnet with requirements shown?

- Given the same network and requirements as in Sample Exercise 1 develop a subnetting scheme using VLSM, given:
 - netA: must support 14 hosts
 - netB: must support 28 hosts
 - netC: must support 2 hosts
 - netD: must support 7 hosts
 - netE: must support 28 host
- Determine what mask allows the required number of hosts.
 - netA: requires a /28 (255.255.255.240) mask to support 14 hosts
 - netB: requires a /27 (255.255.255.224) mask to support 28 hosts
 - netC: requires a /30 (255.255.255.252) mask to support 2 hosts
 - netD: requires a /28 (255.255.255.240) mask to support 7 hosts
 - netE: requires a /27 (255.255.255.224) mask to support 28 hosts

Variable Length Subnet Mask(VLSM)

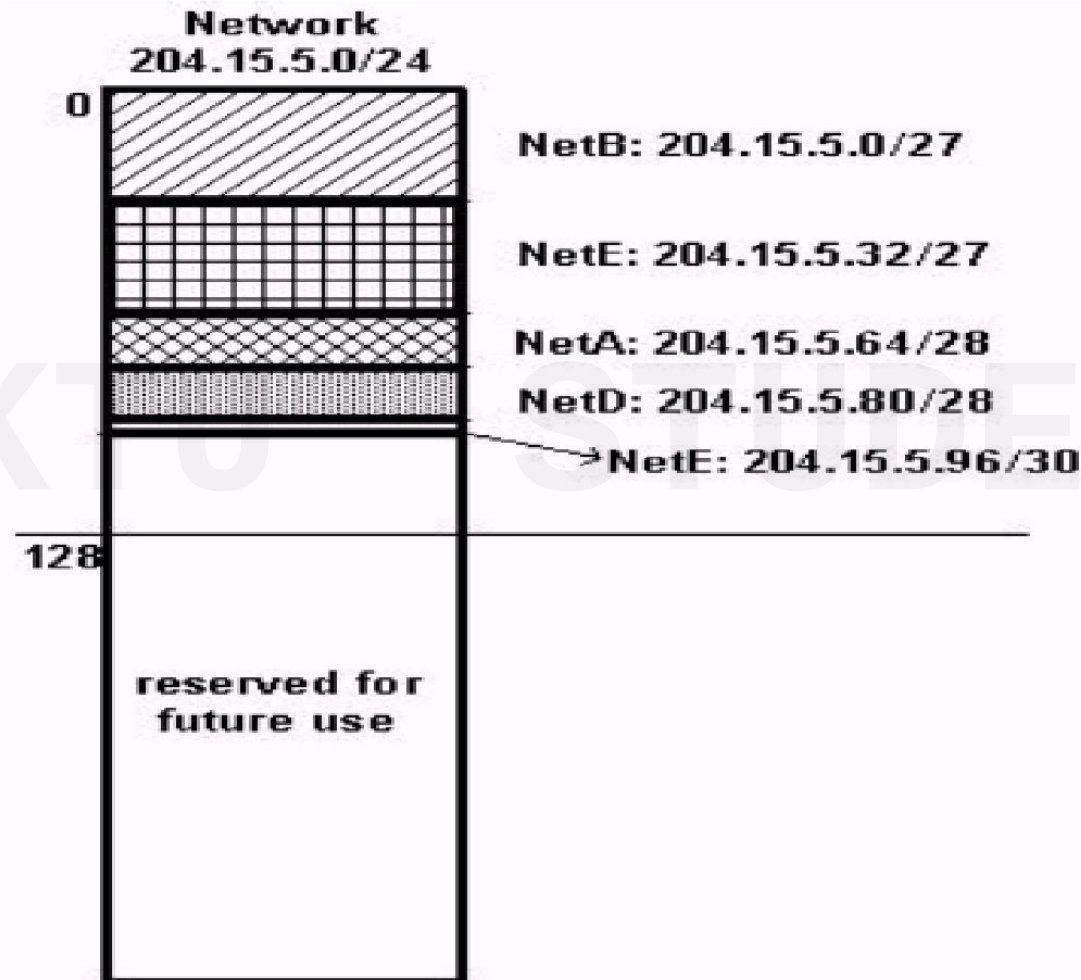
Subnet with requirements shown?

- The easiest way to assign the subnets is to assign the largest first. For example, you can assign in this manner:
 - netB: 204.15.5.0/27 host address range 1 to 30
 - netE: 204.15.5.32/27 host address range 33 to 62
 - netA: 204.15.5.64/28 host address range 65 to 78
 - netD: 204.15.5.80/28 host address range 81 to 94
 - netC: 204.15.5.96/30 host address range 97 to 98

For more study materials>www.ktustudents.in

Variable Length Subnet Mask(VLSM)

Subnet with requirements shown?



CIDR

- Classless Interdomain Routing
- Improve address space utilization
- Routing scalability in the Internet
- For example, if an ISP owns network 172.16.0.0/16, then the ISP can offer 172.16.1.0/24, 172.16.2.0/24, and so on to customers. Yet, when advertising to other providers, the ISP only needs to advertise 172.16.0.0/16